COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***IS(LAB)***
Instructor	***Ms. Ambreen Gul***
Assignment No.	***04***

Student Details

Registration No.	***Name***
FA24-BSE-013	***Ahmad Raza***

Task 1

Code Screenshots

Task2.py U | task_3.py U | Activity_1.java U | CSP_colors U | Assignment_3_Task_1.py U | Assignment_4_task_1.py U

SE-4-VScode > IS Lab > Assignment_4_task_1.py > encrypt

```python
# Simple RSA implementation (for learning only)
# Step 1: Choose two small prime numbers
p = 11
q = 13
# Step 2: Calculate n = p * q
n = p * q
# Step 3: Calculate Euler's Totient function
phi = (p - 1) * (q - 1)
# Step 4: Choose e (public key exponent)
e = 7   # must be coprime with phi
# Step 5: Find d (private key exponent)
# d is modular inverse of e mod phi
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
d = mod_inverse(e, phi)
print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
# -------- ENCRYPTION FUNCTION --------
def encrypt(message, e, n):
    cipher = []
    for char in message:
        # Convert character to number (ASCII)
        m = ord(char)
        # RSA formula: c = m^e mod n
        c = pow(m, e, n)
        cipher.append(c)
    return cipher
```

Task2.py U | task_3.py U | Activity_1.java U | CSP_colors U | Assignment_3_Task_1.py U | Assignment_4_task_1.py U

SE-4-VScode > IS Lab > Assignment_4_task_1.py > encrypt

```python
# -------- DECRYPTION FUNCTION --------
def decrypt(cipher, d, n):
    message = ""
    for c in cipher:
        # RSA formula: m = c^d mod n
        m = pow(c, d, n)
        message += chr(m)
    return message
# -------- TEST --------
text = "ALI"   # your name or short string
print("\nOriginal Message:", text)
encrypted = encrypt(text, e, n)
print("Encrypted:", encrypted)
decrypted = decrypt(encrypted, d, n)
print("Decrypted:", decrypted)
```

Code Explanation

This program demonstrates the basic working of the RSA encryption algorithm using small numbers so the mathematical logic is easy to understand. First, two small prime numbers p and q are chosen. These primes are used to calculate n, which is simply their product and becomes part of both the public and private keys. Then, we compute Euler's Totient function phi = (p−1)(q−1), which is important for determining the relationship between the public and private keys. After that, we select a number e such that it is coprime with phi (meaning they share no common factors except 1). This e becomes the public key exponent.

Mod Inverse Function
To find the private key, we create the mod_inverse() function. This function calculates d, which is the modular inverse of e with respect to phi. In simple terms, d is chosen such that (e × d) mod phi = 1. We create this function because finding the modular inverse is a key mathematical step in RSA, and separating it into a function keeps the code clean and reusable instead of repeating the logic.

Encrypt Function
The encrypt() function is used to convert a message into encrypted form using the public key (e, n). Inside this function, each character of the message is first converted into its ASCII number using ord(). Then the RSA formula c = m^e mod n is applied to generate encrypted values. We store all encrypted values in a list. This function is created to clearly separate the encryption logic and allow easy reuse for different messages.

Decrypt Function
The decrypt() function reverses the process using the private key (d, n). It takes each encrypted number and applies the formula m = c^d mod n to recover the original ASCII value, which is then converted back into a character using chr(). This function is necessary to demonstrate how the private key is used to retrieve the original message securely.
Finally, the program tests RSA by encrypting and decrypting a short string (such as a name). The output shows that the encrypted message looks like a list of numbers, and after decryption, the original text is recovered correctly. This demonstrates the core idea of RSA: even though the encryption process is public, only the private key can correctly decrypt the message.

Output Screenshot

```
PS D:\BS-SE 04\SE-4-VScode> & C:\Users\DELL\AppData\Local\Python\bin\python.exe "d:/BS-SE 04/SE-4-VScode/IS Lab/Assignment_4_task_1.p
y"
Public Key (e, n): (7, 143)
Private Key (d, n): (103, 143)

Original Message: ALI
Encrypted: [65, 54, 83]
Decrypted: ALI
PS D:\BS-SE 04\SE-4-VScode>
```

Task 2

Code Screenshots

lors U | Assignment_3_Task_1.py U | Assignment_4_task_1.py U | Assignment_3_Task_2.py U | Assignment_4_Task_2.py U

SE-4-VScode > IS Lab > Assignment_4_Task_2.py > ...

```python
import hashlib
# -------- RSA KEY GENERATION --------
p = 11
q = 13
n = p * q
phi = (p - 1) * (q - 1)
e = 7  # public exponent
# Find modular inverse
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
d = mod_inverse(e, phi)
print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
# -------- HASH FUNCTION --------
def hash_message(message):
    # SHA256 hashing
    hash_obj = hashlib.sha256(message.encode())
    return int(hash_obj.hexdigest(), 16)
# -------- SIGN FUNCTION --------
def sign(message, d, n):
    h = hash_message(message)
    signature = pow(h, d, n)   # sign using private key
    return signature
# -------- VERIFY FUNCTION --------
def verify(message, signature, e, n):
    h = hash_message(message)
    h_from_signature = pow(signature, e, n)  # decrypt signature
```

lors U | Assignment_3_Task_1.py U | Assignment_4_task_1.py U | Assignment_3_Task_2.py U | Assignment_4_Task_2.py U

SE-4-VScode > IS Lab > Assignment_4_Task_2.py > ...

```python
def verify(message, signature, e, n):
    # Compare both hashes
    return h % n == h_from_signature
# -------- TEST --------
message = "Hello"
print("\nOriginal Message:", message)
signature = sign(message, d, n)
print("Signature:", signature)
# Verify correct message
is_valid = verify(message, signature, e, n)
print("Verification (Original):", is_valid)
# -------- MODIFY MESSAGE --------
fake_message = "Hella"   # slight change
print("\nModified Message:", fake_message)
is_valid_fake = verify(fake_message, signature, e, n)
print("Verification (Modified):", is_valid_fake)
```

Explanation of Code

This program demonstrates how a digital signature works using the RSA algorithm and hashing. First, RSA keys are generated by selecting two small prime numbers p and q, then calculating n = p × q and Euler's Totient phi = (p−1)(q−1). A public key exponent e is chosen, and the private key d is computed using the mod_inverse() function. This function is created to calculate the modular inverse of e, which is necessary to form the private key. It is separated into its own function to keep the code organized and reusable, since this calculation is a core part of RSA.

Hash Message Function

Next, the hash_message() function is used to convert the message into a fixed-size hash using SHA256. Instead of signing the entire message directly, we sign its hash because hashing is faster and ensures a fixed-length representation. This function is created to isolate the hashing process, making it easy to reuse and modify if needed. The sign() function creates the digital signature. It first computes the hash of the message, then encrypts that hash using the private key with the formula signature = hash^d mod n. This function is necessary to clearly represent the signing process and to separate it from other parts of the program.

Verify Function

The verify() function checks whether a signature is valid. It hashes the received message again and also decrypts the signature using the public key with the formula hash_from_signature = signature^e mod n. If both hashes match, the signature is valid. This function is created to encapsulate the verification logic and demonstrate how anyone with the public key can verify authenticity without needing the private key. Finally, the program tests the system by signing an original message and verifying it successfully. Then, the message is slightly modified, and verification is attempted again. This time it fails, showing that even a small change in the message produces a completely different hash, which breaks the signature. This demonstrates the key purpose of digital signatures: ensuring both authenticity and data integrity.

Code Output

```
PS D:\BS-SE 04\SE-4-VScode> & C:\Users\DELL\AppData\Local\Python\bin\python.exe "d:/BS-SE 04/SE-4-VScode/IS Lab/Assignment_4_Task_2.p
y"
Public Key (e, n): (7, 143)
Private Key (d, n): (103, 143)

Original Message: Hello
Signature: 30
Verification (Original): True

Modified Message: Hella
Verification (Modified): False
PS D:\BS-SE 04\SE-4-VScode>
```